03 — Backend, Database and Authentication Guide

For: Member 3 — Backend, Database and Authentication Developer **Status:** Implementation handbook. Every schema field, endpoint, env var, and security rule in this document matches `docs/01-MASTER-PROJECT-DEVELOPMENT-GUIDE.md` exactly. If anything here ever appears to disagree with Document 1, Document 1 wins — stop and reconcile before writing code. **Reviewer:** Member 1 (team lead) reviews every PR that touches authentication, the database models, or `app.js` / `server.js`, per Document 1 Section 14.

---

1. Scope

1.1 What you own

Path	What lives there
`server/src/models/`	All seven Mongoose schemas: `User.js`, `Product.js`, `Offer.js`, `PriceObservation.js`, `WatchlistItem.js`, `Alert.js`, `NotificationEvent.js`
`server/src/routes/`	Every route file **except** `monitor.js` (Member 1's)
`server/src/controllers/`	Every controller file **except** the monitor controller (Member 1's)
`server/src/middleware/`	`auth.js`, `validate.js`, `errorHandler.js`
`server/src/config/`	`db.js`
`server/src/app.js`	Express app wiring
`server/src/server.js`	HTTP server bootstrap
`server/src/utils/`	`asyncHandler.js` (you own this one; `money.js` is also here but is primarily used by Member 1's comparison/alert logic — coordinate before changing it)

1.2 What you never touch

- `server/src/providers/` — Member 4's retailer adapters (`flipkart.js`, `fixture.js`, `index.js`). Even though your `routes/products.js` and `routes/offers.js` live in your folder, they call into **services**, not directly into providers — see Section 11.1.
- `server/src/services/notifications/` — Member 4's email/Telegram senders.

1.3 What you coordinate on before touching

- `server/src/services/priceComparison.js`, `server/src/services/priceMonitor.js`, `server/src/services/alertEvaluator.js` — all Member 1's. You will frequently need to call into or be called by this layer (e.g., your `GET /api/products/search` route needs comparison logic; `alertEvaluator.js` reads and writes fields on your `Alert` model). Message Member 1 before changing the shape of anything these files depend on.
- The `Alert` and `Offer` model **fields** specifically — `alertEvaluator.js` reads `armed`, `lastTriggeredAt`, `targetPriceInPaise`, `dropAmountInPaise`, `type`, `status` on `Alert`, and reads `priceInPaise`, `lastStatus`, `lastError` on `Offer`. You own the schema files, but a field rename or type change here breaks Member 1's code silently, so flag it first.

1.4 Routes you own but don't fully implement alone

`routes/monitor.js` is explicitly **not yours** — Member 1 owns it end to end, including its controller. If you create `server/src/routes/` as a folder on Day 1 before Member 1 has started, create an empty placeholder or leave it for them; don't write monitor logic yourself.
The Telegram-linking **route shell** is yours (`POST /api/telegram/link`, and the `POST /api/telegram/link-code` addition — see Section 11.4), but the actual code that talks to the Telegram Bot API lives in `server/src/services/notifications/telegram.js`, which is Member 4's. Your controller calls into that service; you don't write what's inside it.

---

2. Day 1: Windows setup

2.1 Check your tools

Open PowerShell and run:

```
node --version
  npm --version
  git --version
```

You need **Node 20 LTS or newer** (`v20.x.x` or higher) and any reasonably recent Git. If `node --version` prints something below `v20`, stop and install Node 20 LTS from nodejs.org (the "LTS" button, not "Current") before doing anything else — the whole team needs the same major version so nobody hits a "works on my machine" bug from a Node API that changed between versions.

If `git --version` fails entirely, Git isn't installed — see Document 5 Section 2.2 for the Windows installer walkthrough (that's the team lead's document, but the install steps apply to you too).

2.2 Create a MongoDB Atlas account and free cluster

What MongoDB Atlas is, in plain terms: Atlas is MongoDB's own hosted database service. Instead of each of the 4 team members installing and running a MongoDB server on their own Windows laptop (which would mean 4 separate, disconnected databases, and a 5th database to set up again for production), Atlas gives the whole team **one shared database that lives on the internet**, reachable from anyone's laptop and from the deployed backend later. This is why Document 1 Section 3 picked it: zero local install, identical connection experience for all 4 members, and the exact same database continues to work after deployment (Document 1 Section 12).

Walkthrough:

1. Go to mongodb.com/cloud/atlas/register and create a free account (email/password or Google/GitHub sign-in).
2. When prompted to create a deployment, choose the **free tier** — labeled **M0** (sometimes shown as "Free" in a plan picker). Do not pick a paid tier; M0 is sufficient for this entire project.
3. Pick any cloud provider/region close to your team (e.g., AWS Mumbai, `ap-south-1`) — it doesn't need to match your hosting provider.
4. Name the cluster something recognizable, e.g. `priceradar-cluster`.
5. **Create a database user:** Atlas will prompt you to create a username and password for *database access* (this is separate from your Atlas login). Write these down somewhere safe — you'll need them in the connection string. Use a strong, generated password (no special characters that need URL-encoding, like `@` or `/`, to avoid connection-string headaches — letters, numbers, and `-`/`_` are safest).
6. **Network Access:** In the Atlas left sidebar, go to **Network Access** → **Add IP Address** → choose **Allow Access From Anywhere** (`0.0.0.0/0`). This is a deliberate simplification for a 7-day training project, explicitly sanctioned by Document 1 Section 8: the cluster still requires the username/password from step 5, so this isn't "no security," but it is not production-grade and should be called out as such in the final demo (Document 5 covers this).
7. **Get your connection string:** go to **Database** → your cluster → **Connect** → **Drivers** → select **Node.js** and the version shown. Atlas gives you a string like:

```
mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majo
  rity
```

Replace `<username>` and `<password>` with your database user's credentials from step 5, and add your database name after the host, before the `?`:

```
mongodb+srv://myuser:mypassword@cluster0.xxxxx.mongodb.net/priceradar?retryWrites=true&
  w=majority
```

This full string is your `MONGODB_URI` (Section 7.1 of Document 1). Keep it out of any file that gets committed to Git — it goes in `server/.env` only (Section 3.3 below).

Why not install MongoDB locally instead? You could, but then your laptop's database and a teammate's laptop's database would be two different, disconnected data stores — a product one person creates while testing search wouldn't exist for anyone else, and you'd have to re-point everything at a different connection string the moment you deploy. One shared Atlas cluster avoids all of that for the cost of a few minutes of setup.

---

3. Project scaffold

Run these from the **repository root** (the `PriceRadar/` folder Member 1 created and you've cloned — see Document 5 Section 3 if it doesn't exist yet).

3.1 Create the server folder and initialize it

```
# From the repo root
  if (!(Test-Path server)) { mkdir server }
  cd server
  npm init -y
```

`npm init -y` creates a `package.json` with default values, skipping the interactive prompts. You'll edit it by hand next.

3.2 Install dependencies

```
npm install express mongoose dotenv cors bcryptjs jsonwebtoken express-validator
  npm install --save-dev nodemon jest supertest
```

What each package is for:

Package	Purpose
`express`	The web framework — routes, middleware, request/response handling
`mongoose`	Models MongoDB documents as JavaScript schemas/classes, with validation
`dotenv`	Loads `server/.env` into `process.env` at startup
`cors`	Lets the Express app accept cross-origin requests from the React frontend (different port/domain)
`bcryptjs`	Password hashing — pure JavaScript, no native compiler needed on Windows
`jsonwebtoken`	Issues and verifies JWTs for authentication
`express-validator`	Declarative request-body/query validation
`nodemon` (dev)	Restarts the server automatically when you save a file, during development
`jest` (dev)	Test runner
`supertest` (dev)	Sends HTTP requests against your Express app in tests, without needing a real running server/port

Do not add any other package without updating Document 1 Section 3 first (team agreement) — an unplanned dependency on your machine only is a classic source of "works on my machine" bugs for the other three members.

3.3 Configure `package.json`

Open `server/package.json`. Add `"type": "module"` (so you can use `import`/`export` syntax, matching the rest of the codebase) and the three scripts:

```
{
    "name": "server",
    "version": "1.0.0",
    "type": "module",
    "main": "src/server.js",
    "scripts": {
      "start": "node src/server.js",
      "dev": "nodemon src/server.js",
      "test": "jest"
    },
    "dependencies": {
      "bcryptjs": "^2.4.3",
      "cors": "^2.8.5",
      "dotenv": "^16.4.5",
      "express": "^4.19.2",
      "express-validator": "^7.1.0",
      "jsonwebtoken": "^9.0.2",
      "mongoose": "^8.5.0"
    },
    "devDependencies": {
      "jest": "^29.7.0",
      "nodemon": "^3.1.4",
      "supertest": "^7.0.0"
    }
  }
```

(Your actual installed version numbers may differ slightly — that's fine; don't hand-edit the version numbers npm already wrote, just make sure `"type"` and `"scripts"` match the above.)

3.4 Create `server/.env.example` and `server/.env`

Create `server/.env.example` (this **is** committed to Git — placeholders only, per Document 1 Section 7.3):

```
PORT=5000
  NODE_ENV=development
  MONGODB_URI=mongodb+srv://<username>:<password>@<cluster-host>/priceradar
  JWT_SECRET=replace-with-a-long-random-string
  JWT_EXPIRES_IN=7d
  CLIENT_ORIGIN=http://localhost:5173

  # Member 4's fields — leave blank until Flipkart approval arrives (Document 1 Section
  10)
  FLIPKART_AFFILIATE_ID=
  FLIPKART_AFFILIATE_TOKEN=

  # Member 4's fields — fill in once Brevo/Telegram are set up (Document 4)
  BREVO_API_KEY=
  EMAIL_FROM=
  TELEGRAM_BOT_TOKEN=

  MONITOR_TRIGGER_SECRET=replace-with-a-long-random-string
  MONITORING_ENABLED=true
```

Now create your real, personal `server/.env` (this is **never** committed — copy the example and fill in real values):

```
Copy-Item .env.example .env
```

Then edit `server/.env` by hand:

- `MONGODB_URI` → your real Atlas connection string from Section 2.2.
- `JWT_SECRET` and `MONITOR_TRIGGER_SECRET` → generate real random strings:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Run this twice (once per variable) and paste each result in.

- `CLIENT_ORIGIN` → `http://localhost:5173` (Vite's default dev port) for local work.
- `FLIPKART_AFFILIATE_ID` / `FLIPKART_AFFILIATE_TOKEN` → leave blank. Member 4 fills these in only once Flipkart approval actually arrives (Document 1 Section 10.1) — the fixture provider works with these blank.
- `BREVO_API_KEY`, `EMAIL_FROM`, `TELEGRAM_BOT_TOKEN` → **Member 4's to fill in**, but you still need *some* value (even an empty string, which `dotenv` already gives you by default if the key is simply missing) so your server doesn't crash when it starts up. Your `app.js` / `server.js` code must never assume these are populated — Member 4's services are responsible for their own graceful fallback

when a key is missing (e.g., logging "Telegram not configured" instead of crashing), but you should still confirm your server **starts successfully** even with these blank, since you'll be developing and testing auth/watchlist/alerts long before Member 4's notification pieces are wired in.

Why your server must tolerate blank notification credentials: if `app.js` or any route file you own does something like `if (!process.env.BREVO_API_KEY) throw new Error(...)` at import time, your entire API becomes unusable the moment that variable is empty — which it will be for most of Day 1–4. Only the notification *service* files (Member 4's, which you don't touch) should decide what to do when their own credentials are missing, typically by returning a `status: "failed"` `NotificationEvent` with a clear `errorMessage` rather than crashing the process.

3.5 Create `server/.gitignore`

```
node_modules/
  .env
  *.log
```

If `server/.gitignore` doesn't exist yet (Member 1 may have created a root-level one in Document 5 Section 3.3 that already covers this), check first — don't create a conflicting duplicate. A root `.gitignore` with `node_modules/`, `.env`, and `*.log` already covers `server/` too, since `.gitignore` patterns apply recursively unless anchored with a leading `/`.

---

4. Folder structure

This is the final `server/` tree from Document 1 Section 4. Build exactly this — don't rename folders.

```
server/
  ├── src/
  │   ├── config/              # ← YOU: db.js (Mongoose connection)
  │   ├── models/              # ← YOU: all 7 schemas
  │   ├── routes/              # ← YOU (except monitor.js, Member 1's)
  │   ├── controllers/         # ← YOU (except the monitor controller, Member 1's)
  │   ├── middleware/          # ← YOU: auth.js, errorHandler.js, validate.js
  │   ├── services/
  │   │   ├── priceComparison.js   # Member 1 — do not edit
  │   │   ├── priceMonitor.js      # Member 1 — do not edit
  │   │   ├── alertEvaluator.js    # Member 1 — coordinate before editing
  │   │   └── notifications/       # Member 4 — DO NOT TOUCH
  │   ├── providers/            # Member 4 — DO NOT TOUCH
  │   ├── jobs/                 # Member 4 — local-dev convenience only, DO NOT TOUCH
  │   ├── utils/                # ← YOU: asyncHandler.js (money.js exists here too — sh
  ared, coordinate on changes)
  │   ├── app.js                # ← YOU
  │   └── server.js             # ← YOU
  ├── tests/                    # ← YOU (your own test files; Member 4 adds provider/no
  tification tests here too)
  ├── package.json
  └── .env.example
```

Create the folders you need now, even empty (Git doesn't track empty folders, so add a placeholder `.gitkeep` file if you want them to show up before real files exist):

```
cd server/src
  mkdir config, models, routes, controllers, middleware, utils -Force
  cd ../..
```

(Leave `services/`, `providers/`, `jobs/` for Members 1 and 4 to create — don't create empty versions of folders you don't own unless the scaffold task in Document 5 explicitly asks for placeholders.)

---

5. Database connection (`server/src/config/db.js`)

5.1 Complete code

```
// server/src/config/db.js
  import mongoose from 'mongoose';

  export async function connectDB() {
    const uri = process.env.MONGODB_URI;

    if (!uri) {
      throw new Error(
        'MONGODB_URI is not set. Copy server/.env.example to server/.env and fill in a
  real Atlas connection string.'
      );
    }

    mongoose.connection.on('connected', () => {
      console.log('[db] Mongoose connected to MongoDB Atlas');
    });

    mongoose.connection.on('error', (err) => {
      console.error('[db] Mongoose connection error:', err.message);
    });

    mongoose.connection.on('disconnected', () => {
      console.warn('[db] Mongoose disconnected from MongoDB');
    });

    await mongoose.connect(uri);
  }

  export default connectDB;
```

5.2 Beginner explanation

`async` / `await` **, in plain terms:** connecting to a database over the network takes time — it's not instant like adding two numbers. `mongoose.connect(uri)` returns a **Promise**, a placeholder for "the connection will finish at some point, successfully or with an error." `await` pauses this function (and only this function — the rest of your program keeps running) until that Promise resolves. `async` on the

function definition is required any time you want to use `await` inside it. Without `await`, `connectDB()` would return immediately, before the connection was actually ready, and the very first database query in your app could fail with a confusing "not connected" error.

Why Mongoose instead of the raw MongoDB driver: MongoDB itself will happily store any shape of document you hand it — there's no built-in concept of "every user document must have an email." Mongoose sits on top of the raw driver and adds **schemas**: you declare up front what fields a `User` document should have, their types, which are required, and simple validation rules (an email must look like an email, a number must be an integer, etc.). Mongoose then enforces this automatically on every `save()` / `create()`, and rejects a bad write with a clear `ValidationError` instead of silently storing garbage. For a team of 4 beginners, that safety net is worth far more than the small amount of extra ceremony.

The connection events: `mongoose.connection` is an `EventEmitter` — an object that announces things happening on it (e.g. "my connection state just changed") by calling any functions you've registered for that happening. `.on('connected', fn)` means "whenever the connection state becomes `connected`, run `fn`." These three listeners (`connected`, `error`, `disconnected`) exist purely so you get a visible `console.log` / `console.error` line in your terminal that tells you what's actually happening to the database connection — invaluable when something goes wrong, instead of a silent hang.

Why throw if `MONGODB_URI` is missing: failing loudly and immediately, with a message that tells the developer exactly what to do (copy `.env.example`, fill in a real value), is far better than the server limping along and failing mysteriously on the first database query three screens of logs later.

---

6. All 7 Mongoose models

Every field name, type, `required`/optional status, `enum`, and index below matches Document 1 Section 5 **exactly**. Read that section again before writing code if anything here looks ambiguous.

The paise-money policy (brief recap — see Document 1 Section 5 for the full "why"): every price is stored as an **integer number of paise** (`rupees × 100`, rounded) in a field named `priceInPaise`, never as a JavaScript float. `₹14,999.00` is stored as the integer `1499900`. This sidesteps classic floating-point bugs (`0.1 + 0.2 !== 0.3` in JavaScript) when the system later does price-drop arithmetic (`previousPriceInPaise - currentPriceInPaise >= dropAmountInPaise`). Convert to rupees only at the point of display: `priceInPaise / 100`.

6.1 Mongoose schema syntax, beginner-level

Before the models, here's what the syntax you're about to write means:

- `required: true` — Mongoose refuses to save the document if this field is missing, and throws a `ValidationError` naming the field.
- `enum: [...]` — the field's value must be one of exactly these strings (or numbers); anything else is rejected at validation time, before it ever reaches the database.

- `unique: true` (inside a field, or via `schema.index({...}, {unique: true})`) — MongoDB creates an index that physically refuses a second document with the same value(s), returning a duplicate-key error your controller must catch and translate into a `409 CONFLICT` (Section 10.3 shows this for registration).
- `timestamps: true` (a schema **option**, not a field) — Mongoose automatically adds and maintains `createdAt` and `updatedAt` `Date` fields on every document, updated on every save, with zero extra code from you.
- `type: mongoose.Schema.Types.ObjectId, ref: 'ModelName'` — this field stores another document's `_id` (a reference, like a foreign key in a relational database), and `ref` tells Mongoose which model to look in if you ever call `.populate()` to fetch the related document automatically instead of just its id.

6.2 `User` (`server/src/models/User.js`)

```
// server/src/models/User.js
  import mongoose from 'mongoose';

  const userSchema = new mongoose.Schema(
    {
      name: {
        type: String,
        required: true,
        trim: true,
      },
      email: {
        type: String,
        required: true,
        unique: true,
        lowercase: true,
        trim: true,
        match: [/^[^\s@]+@[^\s@]+.[^\s@]+$/, 'Please provide a valid email address'],
      },
      passwordHash: {
        type: String,
        required: true,
        // Never select this field by default so a stray `User.find()` elsewhere
        // in the codebase can't accidentally leak hashes. Explicitly
        // `.select('+passwordHash')` when you actually need it (login).
        select: false,
      },
      telegramChatId: {
        type: String,
        required: false,
        default: null,
      },
    },
    { timestamps: true }
  );

  const User = mongoose.model('User', userSchema);
  export default User;
```

Note the `select: false` on `passwordHash` — this is an extra safety net beyond "just remember not to return it in the controller" (Section 12 shows the explicit `.select('+passwordHash')` needed at login, and the explicit exclusion elsewhere).

6.3 `Product` (`server/src/models/Product.js`)

```
// server/src/models/Product.js
  import mongoose from 'mongoose';

  const productSchema = new mongoose.Schema(
    {
      title: {
        type: String,
        required: true,
        trim: true,
      },
      brand: {
        type: String,
        required: false,
      },
      model: {
        type: String,
        required: false,
      },
      variant: {
        type: String,
        required: false,
      },
      canonicalKey: {
        type: String,
        required: true,
        unique: true,
        lowercase: true,
      },
      imageUrl: {
        type: String,
        required: false,
      },
    },
    { timestamps: { createdAt: true, updatedAt: false } }
  );

  const Product = mongoose.model('Product', productSchema);
  export default Product;
```

`canonicalKey` is the normalized slug (e.g. `samsung-galaxy-m14-5g-128gb-ice-blue`) that Member 4's provider-normalization step computes to avoid creating a second `Product` document for the same

real-world item. You own this model file, but Member 4's code is what actually creates these documents during provider ingestion — don't add fields here without telling them.

6.4 `Offer` (`server/src/models/Offer.js`)

```
// server/src/models/Offer.js
  import mongoose from 'mongoose';

  const offerSchema = new mongoose.Schema(
    {
      productId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Product',
        required: true,
      },
      provider: {
        type: String,
        enum: ['flipkart', 'amazon'],
        required: true,
      },
      externalProductId: {
        type: String,
        required: true,
      },
      title: {
        type: String,
        required: true,
      },
      productUrl: {
        type: String,
        required: true,
      },
      priceInPaise: {
        type: Number,
        required: true,
        // Enforced as an integer — never a float. See Document 1 Section 5.
        validate: {
          validator: Number.isInteger,
          message: 'priceInPaise must be an integer (rupees x 100, rounded)',
        },
      },
      currency: {
        type: String,
        required: true,
        default: 'INR',
      },
```

```
    availability: {
      type: String,
      enum: ['in_stock', 'out_of_stock', 'unknown'],
      required: true,
    },
    isSample: {
      type: Boolean,
      required: true,
      default: true,
    },
    lastFetchedAt: {
      type: Date,
      required: false,
    },
    lastSuccessAt: {
      type: Date,
      required: false,
    },
    lastStatus: {
      type: String,
      enum: ['ok', 'stale', 'error'],
      required: true,
      default: 'stale',
    },
    lastError: {
      type: String,
      required: false,
    },
  },
  { timestamps: true }
);

// A real listing is uniquely identified by (provider, externalProductId) —
// the same retailer never assigns the same product id to two different
// listings, so this compound unique index stops duplicate Offer documents.
offerSchema.index({ provider: 1, externalProductId: 1 }, { unique: true });

const Offer = mongoose.model('Offer', offerSchema);
export default Offer;
```

6.5 `PriceObservation` (`server/src/models/PriceObservation.js`)

```
// server/src/models/PriceObservation.js
  import mongoose from 'mongoose';

  const priceObservationSchema = new mongoose.Schema({
    offerId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Offer',
      required: true,
    },
    priceInPaise: {
      type: Number,
      required: function () {
        // Only required when the fetch succeeded — a failed fetch has no price.
        return this.status === 'success';
      },
      validate: {
        validator: function (v) {
          return v === undefined || v === null || Number.isInteger(v);
        },
        message: 'priceInPaise must be an integer',
      },
    },
    status: {
      type: String,
      enum: ['success', 'failure'],
      required: true,
    },
    errorMessage: {
      type: String,
      required: false,
    },
    fetchedAt: {
      type: Date,
      required: true,
      default: Date.now,
    },
  });

  // Fast "most recent observation(s) for this offer" queries, and the exact
  // index Document 1 Section 5.4 calls for (offerId + fetchedAt together).
  priceObservationSchema.index({ offerId: 1, fetchedAt: -1 });
```

```
const PriceObservation = mongoose.model('PriceObservation', priceObservationSchema);
export default PriceObservation;
```

Note: `PriceObservation` has no `timestamps: true` — it uses its own explicit `fetchedAt` field instead, because that's the semantically meaningful timestamp here (when the price was observed), not "when this document row was created" (though in practice they'll usually be the same moment).

6.6 `WatchlistItem` (`server/src/models/WatchlistItem.js`)

```
// server/src/models/WatchlistItem.js
import mongoose from 'mongoose';

const watchlistItemSchema = new mongoose.Schema(
  {
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
    offerId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Offer',
      required: true,
    },
  },
  { timestamps: { createdAt: true, updatedAt: false } }
);

// A user cannot watch the same offer twice.
watchlistItemSchema.index({ userId: 1, offerId: 1 }, { unique: true });

const WatchlistItem = mongoose.model('WatchlistItem', watchlistItemSchema);
export default WatchlistItem;
```

6.7 `Alert` (`server/src/models/Alert.js`)

```
// server/src/models/Alert.js
  import mongoose from 'mongoose';

  const alertSchema = new mongoose.Schema(
    {
      userId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true,
      },
      offerId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Offer',
        required: true,
      },
      type: {
        type: String,
        enum: ['target_price', 'fixed_drop'],
        required: true,
      },
      targetPriceInPaise: {
        type: Number,
        required: function () {
          return this.type === 'target_price';
        },
        validate: {
          validator: function (v) {
            return v === undefined || v === null || Number.isInteger(v);
          },
          message: 'targetPriceInPaise must be an integer',
        },
      },
      dropAmountInPaise: {
        type: Number,
        required: function () {
          return this.type === 'fixed_drop';
        },
        validate: {
          validator: function (v) {
            return v === undefined || v === null || Number.isInteger(v);
          },
```

```
        message: 'dropAmountInPaise must be an integer',
      },
    },
    channels: {
      type: [String],
      enum: ['email', 'telegram'],
      required: true,
      validate: {
        validator: (arr) => Array.isArray(arr) && arr.length >= 1,
        message: 'channels must contain at least one of "email" or "telegram"',
      },
    },
    status: {
      type: String,
      enum: ['active', 'paused'],
      required: true,
      default: 'active',
    },
    armed: {
      type: Boolean,
      required: true,
      default: true,
    },
    lastTriggeredAt: {
      type: Date,
      required: false,
    },
  },
  { timestamps: true }
);

const Alert = mongoose.model('Alert', alertSchema);
export default Alert;
```

Reminder — do not change this file's field names without telling Member 1 first. `alertEvaluator.js` depends on `type`, `targetPriceInPaise`, `dropAmountInPaise`, `armed`, `status`, and `lastTriggeredAt` existing exactly as named here (Document 1 Section 9, Section 14).

6.8 `NotificationEvent` (`server/src/models/NotificationEvent.js`)

```
// server/src/models/NotificationEvent.js
  import mongoose from 'mongoose';

  const notificationEventSchema = new mongoose.Schema(
    {
      userId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true,
      },
      alertId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Alert',
        required: true,
      },
      channel: {
        type: String,
        enum: ['email', 'telegram'],
        required: true,
      },
      status: {
        type: String,
        enum: ['sent', 'failed', 'skipped_duplicate'],
        required: true,
      },
      message: {
        type: String,
        required: true,
      },
      errorMessage: {
        type: String,
        required: false,
      },
      dedupeKey: {
        type: String,
        required: true,
      },
    },
    { timestamps: { createdAt: true, updatedAt: false } }
  );
```

```
// Second, independent safety net against double-sending — see Document 1
// Section 9.3. Member 4's code writes these documents; you own the schema.
notificationEventSchema.index({ alertId: 1, dedupeKey: 1 }, { unique: true });

const NotificationEvent = mongoose.model('NotificationEvent', notificationEventSchem
a);
export default NotificationEvent;
```

You own this model file per Document 1 Section 14, even though Member 4 is the one whose code actually writes these documents (the notification services). Keep the schema in sync with what Member 4's code needs — talk to them before changing it.

---

7. Express app wiring (`server/src/app.js` , `server/src/serve`

`r.js`)

7.1 `server/src/app.js`

```
// server/src/app.js
  import express from 'express';
  import cors from 'cors';

  import authRoutes from './routes/auth.js';
  import productRoutes from './routes/products.js';
  import offerRoutes from './routes/offers.js';
  import watchlistRoutes from './routes/watchlist.js';
  import alertRoutes from './routes/alerts.js';
  import notificationRoutes from './routes/notifications.js';
  import telegramRoutes from './routes/telegram.js';
  import monitorRoutes from './routes/monitor.js'; // owned/implemented by Member 1

  import { errorHandler } from './middleware/errorHandler.js';

  const app = express();

  // CORS: only the configured frontend origin may call this API with
  // credentials/custom headers like Authorization. Never "*" once real JWTs
  // are involved (Document 1 Section 8).
  app.use(
    cors({
      origin: process.env.CLIENT_ORIGIN,
    })
  );

  // Parses incoming JSON request bodies into req.body.
  app.use(express.json());

  // Health check — no auth, used by deployment checks and the team itself.
  app.get('/api/health', (req, res) => {
    res.json({ status: 'ok', time: new Date().toISOString() });
  });

  // Route mounting — every route file's internal paths are relative to its
  // mount point here, e.g. a route defined as `/search` inside products.js
  // becomes `/api/products/search`.
  app.use('/api/auth', authRoutes);
```

```
app.use('/api/products', productRoutes);
app.use('/api/offers', offerRoutes);
app.use('/api/watchlist', watchlistRoutes);
app.use('/api/alerts', alertRoutes);
app.use('/api/notifications', notificationRoutes);
app.use('/api/notification-preferences', notificationRoutes.preferencesRouter ?? noti
ficationRoutes);
app.use('/api/telegram', telegramRoutes);
app.use('/api/monitor', monitorRoutes);

// 404 for anything that matched no route above.
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: { code: 'NOT_FOUND', message: 'Route not found.' },
  });
});

// Centralized error handler — MUST be the last app.use() call. See Section
// 7.3 for why ordering matters here.
app.use(errorHandler);

export default app;
```

A note on the `/api/notification-preferences` mount line above: *Document 1 Section 6.5 lists `/api/notifications` (history) and `/api/notification-preferences` (preferences) as two separate route groups. The simplest, least error-prone approach — and the one used in the worked routes below (Section 11.3) — is two separate route files mounted at two separate paths, rather than one shared router object with a confusing secondary export. Replace the awkward `??` line above with a second, explicit import/mount once you've created `routes/notificationPreferences.js`:*

```
> import notificationPreferencesRoutes from './routes/notificationPreferences.js';
> // ...
> app.use('/api/notification-preferences', notificationPreferencesRoutes);
>
```

Delete the `app.use('/api/notification-preferences', notificationRoutes.preferencesRouter ?? notificationRoutes)` line entirely once you've done this — it was only shown above to make the "don't silently merge two different concerns into one file" point concrete. Section 11.3's task list gives you the real, final two-file version.

7.2 `server/src/server.js`

```
// server/src/server.js
  import 'dotenv/config';
  import app from './app.js';
  import connectDB from './config/db.js';

  const PORT = process.env.PORT || 5000;

  async function start() {
    try {
      await connectDB();
      app.listen(PORT, () => {
        console.log(`[server] PriceRadar API listening on port ${PORT}`);
      });
    } catch (err) {
      console.error('[server] Failed to start:', err.message);
      process.exit(1);
    }
  }

  start();
```

`import 'dotenv/config'` must be the very first line executed in the whole process — it reads `server/.env` and populates `process.env` before any other code (including `connectDB()`, which reads `process.env.MONGODB_URI`) runs.

7.3 Beginner explanation: Express middleware ordering

Express processes `app.use(...)` calls **in the order you write them**, for every incoming request. Each one is a checkpoint the request passes through (or doesn't) on its way to a response:

1. `cors(...)` runs first — it decides whether this request's origin is even allowed to talk to the API at all.
2. `express.json()` runs next — it reads the raw request body and, if it's JSON, parses it into `req.body` so every later handler can just read `req.body.email` instead of parsing raw bytes itself.
3. Your route handlers (`app.use('/api/auth', authRoutes)`, etc.) run next, in the order mounted — Express checks each one's path pattern against the incoming URL and runs the first one that matches.
4. The 404 handler only runs if **nothing above it matched** — if some route already sent a response, Express never reaches this line for that request.
5. `errorHandler` is different from the others: it has **four** parameters (`(err, req, res, next)` — see Section 13.1), which is Express's built-in signal that "this is an error-handling middleware, not a normal one." Express skips straight to the first matching error handler whenever any earlier

middleware or route calls `next(err)` or throws inside an `async` handler wrapped by `asyncHandler` (Section 13.2).

Why the error handler must be last: if you registered it *before* your routes, it would never see any errors those routes throw, because Express only routes an error to handlers registered **after** the point where the error occurred. Put it after every route, with nothing meaningful after it, and it reliably catches everything.

---

8. Authentication middleware (`server/src/middleware/auth.js`)

8.1 Complete code

```
// server/src/middleware/auth.js
  import jwt from 'jsonwebtoken';

  export function requireAuth(req, res, next) {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({
        success: false,
        error: { code: 'AUTH_REQUIRED', message: 'Authentication is required for this r
  oute.' },
      });
    }

    const token = authHeader.slice('Bearer '.length).trim();

    try {
      const payload = jwt.verify(token, process.env.JWT_SECRET);
      req.user = { id: payload.sub };
      return next();
    } catch (err) {
      return res.status(401).json({
        success: false,
        error: { code: 'AUTH_INVALID', message: 'Your session is invalid or has expire
  d. Please log in again.' },
      });
    }
  }
```

8.2 Beginner explanation: what a JWT is

A **JWT (JSON Web Token)** is a compact, URL-safe string with three dot-separated parts: a header, a payload, and a signature — e.g. `eyJhbGc...header.eyJzdWI...payload.SflKxw...signature`. The payload is just JSON (here, something like `{ sub: "<userId>", iat: ..., exp: ... }` — `sub` is the JWT standard's name for "subject," i.e. who this token is about). Anyone can **read** a JWT's payload (it's only base64-encoded, not encrypted) — the security comes entirely from the **signature**.

When your server issues a token at login (Section 12.3), it signs the payload using `JWT_SECRET` — a long random string only your server knows. `jwt.verify(token, process.env.JWT_SECRET)` recomputes that signature from the token's header+payload and checks it matches the signature attached to the token. If anyone tampers with the payload (e.g., changing which user id it claims to be), the signature no longer matches, and `jwt.verify` throws — which is exactly the `catch` block above, returning `401 AUTH_INVALID`. `jwt.verify` also automatically rejects an **expired** token (based on the `exp` claim it embeds from `JWT_EXPIRES_IN` at issue time) with the same kind of error, caught the same way.

This is why a client sends the token back on every request in an `Authorization: Bearer <token>` header rather than, say, a plain `userId` in the request body: a plain id can be typed in by anyone; a valid signed JWT can only have been produced by your server, for the user it says it's for, at login time.

8.3 Attaching `req.user`

`req.user = { id: payload.sub }` is the single most important line for the rest of this document: **every** other route that needs to know "which user is making this request" reads `req.user.id`, never anything the client sent in the body or query string. Section 12 covers how `payload.sub` gets set to the user's real database `_id` at login/register time.

---

9. Validation middleware (`server/src/middleware/validate.js`)

9.1 Complete code

```
// server/src/middleware/validate.js
  import { validationResult } from 'express-validator';

  export function validate(req, res, next) {
    const result = validationResult(req);

    if (!result.isEmpty()) {
      const firstError = result.array()[0];
      return res.status(400).json({
        success: false,
        error: {
          code: 'VALIDATION_ERROR',
          message: firstError.msg,
        },
      });
    }

    return next();
  }
```

9.2 How it's used

`express-validator`'s rule functions (`body('email').isEmail()`, etc.) don't reject a request by themselves — they just **record** whether each rule passed, attached to `req`. `validate` is the one shared middleware that actually reads those recorded results and turns the first failure into the standard error envelope. Every route that accepts a body is wired as:

```
router.post('/some-route', [ /* ...validation rules... */ ], validate, controllerFuncti
  on);
```

Express runs the array of validation rules first (each one checks its own field and moves on, even if it fails — nothing stops the request yet), then `validate` (which actually stops the request with a `400` if anything failed), then finally your controller — which, by the time it runs, can trust that `req.body` already satisfies every rule you declared.

---

10. Worked example: Auth (`server/src/routes/auth.js`, `server/src/controllers/authController.js`)

This is the one feature given to you fully implemented, end to end, as a model for the task-list features in Section 11.

10.1 `server/src/utils/asyncHandler.js` (needed first)

```
// server/src/utils/asyncHandler.js
  export function asyncHandler(fn) {
    return function (req, res, next) {
      Promise.resolve(fn(req, res, next)).catch(next);
    };
  }
```

(Full explanation in Section 13.2 — you need this utility before the controller below will make sense.)

10.2 `server/src/controllers/authController.js`

```
// server/src/controllers/authController.js
  import bcrypt from 'bcryptjs';
  import jwt from 'jsonwebtoken';
  import User from '../models/User.js';
  import { asyncHandler } from '../utils/asyncHandler.js';

  function toPublicUser(userDoc) {
    return {
      id: userDoc._id.toString(),
      name: userDoc.name,
      email: userDoc.email,
      telegramChatId: userDoc.telegramChatId ?? null,
    };
  }

  function signToken(userId) {
    return jwt.sign({ sub: userId.toString() }, process.env.JWT_SECRET, {
      expiresIn: process.env.JWT_EXPIRES_IN,
    });
  }

  export const register = asyncHandler(async (req, res) => {
    const { name, email, password } = req.body;

    const existing = await User.findOne({ email: email.toLowerCase() });
    if (existing) {
      return res.status(409).json({
        success: false,
        error: { code: 'CONFLICT', message: 'An account with this email already exist
  s.' },
      });
    }

    const passwordHash = await bcrypt.hash(password, 10);
    const user = await User.create({ name, email, passwordHash });

    return res.status(201).json({
      success: true,
      data: { user: { id: user._id.toString(), name: user.name, email: user.email } },
    });
  });
```

```
export const login = asyncHandler(async (req, res) => {
  const { email, password } = req.body;

  // passwordHash has `select: false` on the schema, so it must be requested
  // explicitly here — this is the one place in the whole codebase that does.
  const user = await User.findOne({ email: email.toLowerCase() }).select('+passwordHa
sh');

  // Deliberately the SAME generic message whether the email doesn't exist or
  // the password is wrong — per Document 1 Section 13.6, the UI/API must
  // never reveal which of the two was incorrect.
  const genericError = {
    success: false,
    error: { code: 'AUTH_INVALID', message: 'Invalid email or password.' },
  };

  if (!user) {
    return res.status(401).json(genericError);
  }

  const passwordMatches = await bcrypt.compare(password, user.passwordHash);
  if (!passwordMatches) {
    return res.status(401).json(genericError);
  }

  const token = signToken(user._id);

  return res.status(200).json({
    success: true,
    data: {
      token,
      user: { id: user._id.toString(), name: user.name, email: user.email },
    },
  });
});

export const getMe = asyncHandler(async (req, res) => {
  const user = await User.findById(req.user.id);

  if (!user) {
    return res.status(404).json({
      success: false,
```

```
      error: { code: 'NOT_FOUND', message: 'User not found.' },
    });
  }

  return res.status(200).json({ success: true, data: { user: toPublicUser(user) } });
});
```

10.3 `server/src/routes/auth.js`

```
// server/src/routes/auth.js
  import { Router } from 'express';
  import { body } from 'express-validator';
  import { register, login, getMe } from '../controllers/authController.js';
  import { validate } from '../middleware/validate.js';
  import { requireAuth } from '../middleware/auth.js';

  const router = Router();

  router.post(
    '/register',
    [
      body('name').trim().notEmpty().withMessage('Name is required.'),
      body('email').isEmail().withMessage('A valid email is required.').normalizeEmail
  (),
      body('password')
        .isLength({ min: 8 })
        .withMessage('Password must be at least 8 characters long.'),
    ],
    validate,
    register
  );

  router.post(
    '/login',
    [
      body('email').isEmail().withMessage('A valid email is required.').normalizeEmail
  (),
      body('password').notEmpty().withMessage('Password is required.'),
    ],
    validate,
    login
  );

  router.get('/me', requireAuth, getMe);

  export default router;
```

10.4 Why `register` never returns `passwordHash`

Look closely at `register`'s response: it builds a brand-new object (`{ id, name, email }`) by hand rather than returning the Mongoose document directly. This is deliberate and is the pattern to copy everywhere — never do `res.json({ user })` with a raw Mongoose document, even though `passwordHash` has `select: false` and so usually wouldn't be included anyway (don't rely on that alone; be explicit).

---

11. Remaining routes — task list (BE-01 through BE-09)

The tasks below use the full 16-part template (A–P) from the project brief (MASTER-PROMPT-SOURCE.md Section 13). Section G (Implementation) gives real, runnable controller code for every task — not pseudocode.

BE-01 — Database connection and project scaffold

A. Task ID and title: `BE-01` — Configure MongoDB Atlas connection and base server scaffold

B. Owner and reviewer: Owner: Member 3. Reviewer: Member 1. Dependencies: none (first backend task).

C. Purpose: Establish the one shared database connection every other backend feature depends on, and the base `server/` project skeleton.

D. Prerequisites: Node 20 LTS installed (Section 2.1); MongoDB Atlas account, cluster, database user, and connection string ready (Section 2.2).

E. Exact files: Create `server/package.json`, `server/.env.example`, `server/.env` (not committed), `server/.gitignore`, `server/src/config/db.js`.

F. Commands: See Section 3.1–3.5 in full.

G. Implementation: Follow Sections 3 and 5 of this document exactly — `package.json` scripts/type, `.env` / `.env.example` contents, and the complete `db.js` code given in Section 5.1.

H. Beginner explanation: See Section 5.2.

I. Environment configuration: `MONGODB_URI`, `PORT`, `NODE_ENV` (Document 1 Section 7.1).

J. Testing: Create a throwaway `server/src/scratch-test-connection.js`:

```
import 'dotenv/config';
  import connectDB from './config/db.js';
  connectDB().then(() => { console.log('OK'); process.exit(0); });
```

Run `node src/scratch-test-connection.js` from `server/`. Delete this file once confirmed — it's not part of the real app.

K. Expected results: Console prints `[db] Mongoose connected to MongoDB Atlas` then `OK`, and the process exits cleanly (code 0).

L. Troubleshooting: See Section 15 ("MongooseServerSelectionError").

M. Acceptance criteria: `node src/scratch-test-connection.js` succeeds without throwing; `server/.env` is confirmed absent from `git status` output (i.e., it's actually ignored).

N. Git workflow:

```
git switch main
  git pull --ff-only origin main
  git switch -c feature/project-scaffold
  git add server/package.json server/package-lock.json server/.env.example server/.gitignore server/src/config/db.js
  git status
  git diff --cached
  git commit -m "feat(server): scaffold Express project and MongoDB Atlas connection"
  git push -u origin feature/project-scaffold
```

Open a PR on GitHub, request review from Member 1.

O. Completion checklist:

- [ ] `server/.env` is NOT in `git status` output
- [ ] `server/.env.example` has every variable from Document 1 Section 7.1, placeholders only
- [ ] `npm run dev` starts without error (once `server.js` / `app.js` exist from BE-03)

P. AI-assisted debugging prompt: "Here is my `server/src/config/db.js` and the exact error message/stack trace I'm getting when I run `node src/scratch-test-connection.js`. Inspect the code and the error, identify the root cause, suggest the smallest safe change, and tell me how to verify the fix without changing anything unrelated."

BE-02 — All seven Mongoose models

A. Task ID and title: `BE-02` — Implement all 7 database models

B. Owner and reviewer: Owner: Member 3. Reviewer: Member 1 (schema correctness is a shared contract — Document 1 Section 14). Dependencies: BE-01.

C. Purpose: Create the persistence layer every other feature (auth, watchlist, alerts, monitoring) reads and writes.

D. Prerequisites: BE-01 complete; Document 1 Section 5 read in full.

E. Exact files: `server/src/models/User.js`, `Product.js`, `Offer.js`, `PriceObservation.js`, `WatchlistItem.js`, `Alert.js`, `NotificationEvent.js`.

F. Commands: None beyond creating files (`mongoose` already installed in BE-01).

G. Implementation: Use the complete code in Section 6.2–6.8 of this document, verbatim — field names, types, enums, and indexes must match Document 1 Section 5 exactly.

H. Beginner explanation: See Section 6.1 (schema syntax) and the paise-money policy note at the top of Section 6.

I. Environment configuration: None additional.

J. Testing: Write `server/tests/models.test.js`:

```
import mongoose from 'mongoose';
  import User from '../src/models/User.js';
  import Offer from '../src/models/Offer.js';

  describe('User model', () => {
    it('rejects a document with no email', async () => {
      const user = new User({ name: 'Test', passwordHash: 'x' });
      await expect(user.validate()).rejects.toThrow();
    });
  });

  describe('Offer model', () => {
    it('rejects a non-integer priceInPaise', async () => {
      const offer = new Offer({
        productId: new mongoose.Types.ObjectId(),
        provider: 'flipkart',
        externalProductId: 'ABC123',
        title: 'Test offer',
        productUrl: 'https://example.com',
        priceInPaise: 1499900.5,
        availability: 'in_stock',
      });
      await expect(offer.validate()).rejects.toThrow();
    });
  });
```

This uses `.validate()` (schema validation only, no real database write needed), so it runs without a live Atlas connection. See Section 14 for the full test setup.

K. Expected results: `npx jest tests/models.test.js` — both tests pass (the invalid documents correctly throw).

L. Troubleshooting: See Section 15 ("Mongoose ValidationError").

M. Acceptance criteria: Every field name, type, `enum`, and index matches Document 1 Section 5 exactly; `npx jest tests/models.test.js` passes.

N. Git workflow: Continue on `feature/project-scaffold` (models are foundational scaffold work) or open `feature/user-authentication` early if Day 1 is already over — ask Member 1 which branch to use if the timing is ambiguous. Commands follow the same pattern as BE-01 (Section 16 shows the full sequence once).

O. Completion checklist:

- [ ] All 7 model files exist and export a default Mongoose model
- [ ] Compound unique indexes exist on `Offer` (`provider` + `externalProductId`), `WatchlistItem` (`userId` + `offerId`), `NotificationEvent` (`alertId` + `dedupeKey`)
- [ ] `priceInPaise` / `targetPriceInPaise` / `dropAmountInPaise` all validate as integers

P. AI-assisted debugging prompt: "Here is my Mongoose schema file and a `ValidationError` I'm getting when saving a document with this exact data: [paste data]. Inspect the schema and the error, identify which rule is failing and why, suggest the smallest safe fix, and tell me how to verify it."

BE-03 — Express app wiring

A. Task ID and title: `BE-03` — Wire up `app.js` / `server.js`, CORS, health check, and centralized error handling

B. Owner and reviewer: Owner: Member 3. Reviewer: Member 1. Dependencies: BE-01, BE-02.

C. Purpose: Produce a running Express server that the frontend (Member 2) can actually call, with the health check Document 1 Section 17 (ACC-07) requires.

D. Prerequisites: BE-01 and BE-02 complete.

E. Exact files: `server/src/app.js`, `server/src/server.js`, `server/src/middleware/errorHandler.js`, `server/src/utils/asyncHandler.js`.

F. Commands:

```
cd server
  npm run dev
```

G. Implementation: Use Section 7.1–7.2's complete code, Section 13.1's `errorHandler.js`, and Section 10.1's `asyncHandler.js`. Note that `app.js` imports route files that don't exist yet until BE-05 through BE-09 — stub them as empty routers first if you're building this before those tasks:

```
// Temporary stub — replace with the real router in a later task.
  import { Router } from 'express';
  const router = Router();
  export default router;
```

H. Beginner explanation: See Section 7.3 (middleware ordering) and Section 13 (error handling conventions).

I. Environment configuration: `PORT`, `NODE_ENV`, `CLIENT_ORIGIN`.

J. Testing: `npm run dev`, then in a second PowerShell window:

```
curl http://localhost:5000/api/health
```

K. Expected results:

```
{"status":"ok","time":"2026-10-14T09:00:00.000Z"}
```

with HTTP status `200`.

L. Troubleshooting: See Section 15 ("EADDRINUSE", "CORS errors").

M. Acceptance criteria: `GET /api/health` returns `200` with the shape above; the error handler never leaks a stack trace when `NODE_ENV=production` (test by temporarily setting it and hitting a route that throws).

N. Git workflow: `feature/project-scaffold` branch, same pattern as BE-01.

O. Completion checklist:

- [ ] `npm run dev` starts with no errors
- [ ] `/api/health` responds correctly
- [ ] `errorHandler` is the LAST `app.use()` call in `app.js`

P. AI-assisted debugging prompt: "My Express app throws [error] when I call [route]. Here is my `app.js` and `errorHandler.js`. Inspect the middleware order and the error, identify the root cause, suggest the smallest safe fix, and tell me how to verify it."

BE-04 — Auth middleware and validation middleware

A. Task ID and title: `BE-04` — Implement JWT auth middleware and the shared validation wrapper

B. Owner and reviewer: Owner: Member 3. Reviewer: Member 1 (auth-sensitive code — Document 1 Section 14). Dependencies: BE-03.

C. Purpose: Provide the two pieces of middleware every protected and validated route in the whole backend depends on.

D. Prerequisites: BE-03 complete.

E. Exact files: `server/src/middleware/auth.js`, `server/src/middleware/validate.js`.

F. Commands: None beyond what's already installed.

G. Implementation: Section 8.1's complete `auth.js` and Section 9.1's complete `validate.js`, verbatim.

H. Beginner explanation: Section 8.2 (what a JWT is) and Section 9.2 (how validation wiring works).

I. Environment configuration: `JWT_SECRET`.

J. Testing: Covered implicitly by BE-05's auth tests (Section 14) — a route protected by `requireAuth` with no header should return `401 AUTH_REQUIRED`; with a garbage token, `401 AUTH_INVALID`.

K. Expected results:

```
{"success":false,"error":{"code":"AUTH_REQUIRED","message":"Authentication is required
  for this route."}}
```

L. Troubleshooting: See Section 15 ("jwt malformed"/"invalid signature").

M. Acceptance criteria: A request to any protected route with no `Authorization` header returns `401 AUTH_REQUIRED`; a request with an invalid/expired token returns `401 AUTH_INVALID`; a request with a valid token reaches the controller with `req.user.id` set correctly.

N. Git workflow: `feature/user-authentication` branch (see Section 16 for the full worked example).

O. Completion checklist:

- [ ] Missing header → `401 AUTH_REQUIRED`
- [ ] Invalid/expired token → `401 AUTH_INVALID`
- [ ] `req.user.id` is set from the token's `sub` claim, never from client input

P. AI-assisted debugging prompt: "My `requireAuth` middleware isn't attaching `req.user` correctly — here's the middleware code and a route that uses it, plus the actual response I get for this exact request: [details]. Identify the root cause and the smallest safe fix."

BE-05 — Auth routes (worked example, referenced)

A. Task ID and title: `BE-05` — Implement `/api/auth/register`, `/api/auth/login`, `/api/auth/me`

B. Owner and reviewer: Owner: Member 3. Reviewer: Member 1 (auth-sensitive). Dependencies: BE-02, BE-04.

C. Purpose: Let a user create an account, log in, and fetch their own profile — the foundation every per-user feature (watchlist, alerts) builds on.

D. Prerequisites: BE-02 (`User` model), BE-04 (`requireAuth`, `validate`) complete.

E. Exact files: `server/src/routes/auth.js`, `server/src/controllers/authController.js`.

F. Commands: None beyond what's installed.

G. Implementation: This task is fully worked already — see Section 10 of this document for the complete code (Sections 10.2–10.3) and reasoning (Section 10.4). Do not re-derive it; copy it, understand it, and move on.

H. Beginner explanation: Section 10.4 and Section 8.2.

I. Environment configuration: `JWT_SECRET`, `JWT_EXPIRES_IN`.

J. Testing: Full Postman checklist in Section 14.3; automated test in Section 14.2 (`auth.test.js`).

K. Expected results: See the example request/response in Document 1 Section 6.7 (matches `POST /api/alerts`'s pattern; for auth specifically, see Section 10 above).

L. Troubleshooting: Section 15.

M. Acceptance criteria:

- Duplicate email registration → `409 CONFLICT`
- Wrong password OR unknown email at login → identical `401 AUTH_INVALID` message ("Invalid email or password.")
- `GET /api/auth/me` with a valid token never includes `passwordHash`
- Password is bcrypt-hashed in the database (verify directly in Atlas's collection browser, or via a quick script — never log the plaintext password anywhere)

N. Git workflow: `feature/user-authentication` — see Section 16 for the complete worked PowerShell sequence.

O. Completion checklist:

- [ ] All three endpoints match Document 1 Section 6.1 exactly
- [ ] `passwordHash` never appears in any response, in any code path
- [ ] Generic login error message, same for both failure cases

P. AI-assisted debugging prompt: "My `/api/auth/login` returns [actual result] for this request: [request]. Here's `authController.js`. Identify why, propose the smallest safe fix, and tell me how to re-test it."

BE-06 — Product and offer read routes

A. Task ID and title: `BE-06` — Implement `GET /api/products/search`, `GET /api/products/:productId`, `GET /api/offers/:offerId/history`

B. Owner and reviewer: Owner: Member 3. Reviewer: Member 1 (these routes call into Member 1's comparison logic). Dependencies: BE-02; Member 1's `priceComparison.js` (coordinate timing — if it isn't ready yet, build against the raw models directly and swap in the shared service once it exists, per Document 1 Section 19's "contract-first, parallel work" approach).

C. Purpose: Let any visitor (no login required, per Document 1 Section 6.2) search for products and see offers compared side by side.

D. Prerequisites: BE-02 complete; fixture data exists (Member 4's `providers/fixtureData.json`, ingested into `Product`/`Offer` collections — ask Member 4 or Member 1 for the ingestion step's status before testing).

E. Exact files: `server/src/routes/products.js`, `server/src/routes/offers.js`, `server/src/controllers/productController.js`, `server/src/controllers/offerController.js`. Do not touch `server/src/services/priceComparison.js`.

F. Commands: None additional.

G. Implementation:

```
// server/src/controllers/productController.js
  import Product from '../models/Product.js';
  import Offer from '../models/Offer.js';
  import { asyncHandler } from '../utils/asyncHandler.js';

  function toOfferSummary(offer) {
    return {
      id: offer._id.toString(),
      provider: offer.provider,
      title: offer.title,
      priceInPaise: offer.priceInPaise,
      currency: offer.currency,
      availability: offer.availability,
      productUrl: offer.productUrl,
      isSample: offer.isSample,
      lastStatus: offer.lastStatus,
      lastSuccessAt: offer.lastSuccessAt,
    };
  }

  export const searchProducts = asyncHandler(async (req, res) => {
    const q = (req.query.q || '').trim();

    if (!q) {
      return res.status(200).json({ success: true, data: { products: [] } });
    }

    // Simple case-insensitive partial match across title/brand/model for the
    // MVP. A dedicated text index is a reasonable later improvement, not
    // required for the 7-day scope.
    const regex = new RegExp(q.replace(/[.*+?^${}()|[]\]/g, '\$&'), 'i');
    const products = await Product.find({
      $or: [{ title: regex }, { brand: regex }, { model: regex }],
    }).limit(20);

    const productIds = products.map((p) => p._id);
    const offers = await Offer.find({ productId: { $in: productIds } });

    const offersByProduct = new Map();
    for (const offer of offers) {
      const key = offer.productId.toString();
      if (!offersByProduct.has(key)) offersByProduct.set(key, []);
```

```
    offersByProduct.get(key).push(toOfferSummary(offer));
  }

  const result = products.map((p) => ({
    id: p._id.toString(),
    title: p.title,
    brand: p.brand ?? null,
    variant: p.variant ?? null,
    offers: offersByProduct.get(p._id.toString()) ?? [],
  }));

  return res.status(200).json({ success: true, data: { products: result } });
});

export const getProductById = asyncHandler(async (req, res) => {
  const product = await Product.findById(req.params.productId);

  if (!product) {
    return res.status(404).json({
      success: false,
      error: { code: 'NOT_FOUND', message: 'Product not found.' },
    });
  }

  const offers = await Offer.find({ productId: product._id });

  return res.status(200).json({
    success: true,
    data: {
      product: {
        id: product._id.toString(),
        title: product.title,
        brand: product.brand ?? null,
        model: product.model ?? null,
        variant: product.variant ?? null,
        imageUrl: product.imageUrl ?? null,
      },
      offers: offers.map(toOfferSummary),
    },
  });
});
```

```
// server/src/controllers/offerController.js
  import Offer from '../models/Offer.js';
  import PriceObservation from '../models/PriceObservation.js';
  import { asyncHandler } from '../utils/asyncHandler.js';

  export const getOfferHistory = asyncHandler(async (req, res) => {
    const offer = await Offer.findById(req.params.offerId);

    if (!offer) {
      return res.status(404).json({
        success: false,
        error: { code: 'NOT_FOUND', message: 'Offer not found.' },
      });
    }

    const history = await PriceObservation.find({ offerId: offer._id })
      .sort({ fetchedAt: -1 })
      .limit(100);

    return res.status(200).json({
      success: true,
      data: {
        offerId: offer._id.toString(),
        history: history.map((h) => ({
          priceInPaise: h.priceInPaise ?? null,
          status: h.status,
          fetchedAt: h.fetchedAt,
        })),
      },
    });
  });
```

```
// server/src/routes/products.js
  import { Router } from 'express';
  import { searchProducts, getProductById } from '../controllers/productController.js';

  const router = Router();

  router.get('/search', searchProducts);
  router.get('/:productId', getProductById);

  export default router;
```

```
// server/src/routes/offers.js
  import { Router } from 'express';
  import { getOfferHistory } from '../controllers/offerController.js';

  const router = Router();

  router.get('/:offerId/history', getOfferHistory);

  export default router;
```

H. Beginner explanation: These routes have no `requireAuth` — Document 1 Section 6.2 marks them "No" under Auth, because anyone should be able to search and compare prices without an account (only watchlisting/alerting requires login). The regex-escaping line (`q.replace(/[.*+?^${}()|[]\]/g, '\$&')`) exists so a search term containing regex special characters (like `"iPhone (2023)"`) doesn't crash or behave unexpectedly when turned into a `RegExp`.

I. Environment configuration: None additional.

J. Testing: Postman: `GET http://localhost:5000/api/products/search?q=galaxy` with fixture data loaded → expect a `products` array with at least one entry with 2+ offers (Document 1 ACC-01).

K. Expected results: Matches the `OfferSummary`/products shape in Document 1 Section 6.2 exactly.

L. Troubleshooting: Empty results with fixture data present usually means the regex didn't match — double check casing/field names, not a bug in the regex escaping itself.

M. Acceptance criteria: Response shapes match Document 1 Section 6.2 field-for-field; no `requireAuth` on any of these three routes; a 404 for an unknown `productId`/`offerId` uses the standard error envelope.

N. Git workflow: `feature/product-search` branch.

O. Completion checklist:

- [ ] All three routes implemented, no auth required

- [ ] `OfferSummary` shape matches exactly (field names, not similar ones)
- [ ] 404s use the standard error envelope

P. AI-assisted debugging prompt: "My `/api/products/search` returns an empty array even though fixture data exists in MongoDB. Here's my controller and a sample document from the `products` collection. Identify why the query isn't matching and the smallest safe fix."

BE-07 — Watchlist CRUD

A. Task ID and title: `BE-07` — Implement `GET/POST /api/watchlist`, `DELETE /api/watchlist/:watchlistItemId`

B. Owner and reviewer: Owner: Member 3. Reviewer: Member 1. Dependencies: BE-02, BE-04, BE-06 (needs `Offer` documents to watchlist).

C. Purpose: Let a logged-in user save offers to a private list.

D. Prerequisites: Auth working (BE-05); at least one `Offer` document exists to test against.

E. Exact files: `server/src/routes/watchlist.js`, `server/src/controllers/watchlistController.js`.

F. Commands: None additional.

G. Implementation:

```
// server/src/controllers/watchlistController.js
  import mongoose from 'mongoose';
  import WatchlistItem from '../models/WatchlistItem.js';
  import Offer from '../models/Offer.js';
  import { asyncHandler } from '../utils/asyncHandler.js';

  function toOfferSummary(offer) {
    return {
      id: offer._id.toString(),
      provider: offer.provider,
      title: offer.title,
      priceInPaise: offer.priceInPaise,
      currency: offer.currency,
      availability: offer.availability,
      productUrl: offer.productUrl,
      isSample: offer.isSample,
      lastStatus: offer.lastStatus,
      lastSuccessAt: offer.lastSuccessAt,
    };
  }

  export const listWatchlist = asyncHandler(async (req, res) => {
    // CORRECT: always scoped to the authenticated user from the JWT.
    const items = await WatchlistItem.find({ userId: req.user.id }).sort({ createdAt: -
  1 });

    const offerIds = items.map((i) => i.offerId);
    const offers = await Offer.find({ _id: { $in: offerIds } });
    const offerById = new Map(offers.map((o) => [o._id.toString(), o]));

    const result = items
      .map((item) => {
        const offer = offerById.get(item.offerId.toString());
        if (!offer) return null;
        return {
          id: item._id.toString(),
          offer: toOfferSummary(offer),
          createdAt: item.createdAt,
        };
      })
      .filter(Boolean);
```

```
  return res.status(200).json({ success: true, data: { items: result } });
});

export const addToWatchlist = asyncHandler(async (req, res) => {
  const { offerId } = req.body;

  if (!mongoose.Types.ObjectId.isValid(offerId)) {
    return res.status(400).json({
      success: false,
      error: { code: 'VALIDATION_ERROR', message: 'offerId must be a valid id.' },
    });
  }

  const offer = await Offer.findById(offerId);
  if (!offer) {
    return res.status(404).json({
      success: false,
      error: { code: 'NOT_FOUND', message: 'Offer not found.' },
    });
  }

  try {
    const item = await WatchlistItem.create({ userId: req.user.id, offerId });
    return res.status(201).json({
      success: true,
      data: { item: { id: item._id.toString(), offerId, createdAt: item.createdAt }
},
    });
  } catch (err) {
    if (err.code === 11000) {
      // Duplicate key error from the {userId, offerId} unique index.
      return res.status(409).json({
        success: false,
        error: { code: 'CONFLICT', message: 'This offer is already on your watchlis
t.' },
      });
    }
    throw err;
  }
});

export const removeFromWatchlist = asyncHandler(async (req, res) => {
  // CORRECT: deletion is scoped to req.user.id too, so a user can only ever
```

```
    // delete their OWN watchlist item, never one belonging to someone else,
    // even if they guess another user's watchlistItemId.
    const result = await WatchlistItem.findOneAndDelete({
      _id: req.params.watchlistItemId,
      userId: req.user.id,
    });

    if (!result) {
      return res.status(404).json({
        success: false,
        error: { code: 'NOT_FOUND', message: 'Watchlist item not found.' },
      });
    }

    return res.status(200).json({ success: true, data: { deleted: true } });
  });
```

```
// server/src/routes/watchlist.js
  import { Router } from 'express';
  import { body } from 'express-validator';
  import { requireAuth } from '../middleware/auth.js';
  import { validate } from '../middleware/validate.js';
  import { listWatchlist, addToWatchlist, removeFromWatchlist } from '../controllers/wa
  tchlistController.js';

  const router = Router();

  router.use(requireAuth); // every route below requires login

  router.get('/', listWatchlist);

  router.post(
    '/',
    [body('offerId').notEmpty().withMessage('offerId is required.')],
    validate,
    addToWatchlist
  );

  router.delete('/:watchlistItemId', removeFromWatchlist);

  export default router;
```

H. Beginner explanation: `router.use(requireAuth)` applies that middleware to every route defined below it in this file — a shorthand so you don't repeat `requireAuth` on each of the three routes individually. The duplicate-watchlist-item case (`err.code === 11000`) is MongoDB's standard duplicate-key error code, thrown when the `{userId, offerId}` unique index (Section 6.6) is violated — catching it here and turning it into a friendly `409 CONFLICT` is much better than letting it bubble up as an unhandled `500`.

I. Environment configuration: None additional.

J. Testing: See Section 14.3's Postman checklist.

K. Expected results: Matches Document 1 Section 6.3 exactly.

L. Troubleshooting: A `409` on the very first add for a brand-new user usually means `req.user.id` isn't what you think — log it and compare against the actual `userId` field type in the existing documents.

M. Acceptance criteria: A second user cannot see or delete the first user's watchlist items (Document 1 ACC-03) — verify with two different test accounts in Postman.

N. Git workflow: `feature/dashboard-watchlist` branch.

O. Completion checklist:

- [ ] Every query filters by `req.user.id`, never `req.body.userId`
- [ ] Duplicate add → `409 CONFLICT`, not a raw `500`
- [ ] Deleting someone else's item → `404`, not `403` (don't reveal that the item exists for another user)

P. AI-assisted debugging prompt: "My watchlist DELETE route lets a user delete items they don't own / returns the wrong status. Here's the controller. Identify the bug and the smallest safe fix, and give me a Postman test to verify it."

BE-08 — Alerts CRUD

A. Task ID and title: `BE-08` — Implement `GET/POST /api/alerts`, `PATCH /api/alerts/:alertId`, `DELETE /api/alerts/:alertId`

B. Owner and reviewer: Owner: Member 3. Reviewer: Member 1 (this model is read/written by `alertEvaluator.js` — Document 1 Section 14). Dependencies: BE-02, BE-04, BE-06.

C. Purpose: Let a logged-in user define, edit, pause/resume, and delete price alerts.

D. Prerequisites: Auth working; at least one `Offer` to alert on.

E. Exact files: `server/src/routes/alerts.js`, `server/src/controllers/alertController.js`.

F. Commands: None additional.

G. Implementation:

```
// server/src/controllers/alertController.js
  import mongoose from 'mongoose';
  import Alert from '../models/Alert.js';
  import Offer from '../models/Offer.js';
  import { asyncHandler } from '../utils/asyncHandler.js';

  function toPublicAlert(alert) {
    return {
      id: alert._id.toString(),
      offerId: alert.offerId.toString(),
      type: alert.type,
      targetPriceInPaise: alert.targetPriceInPaise ?? null,
      dropAmountInPaise: alert.dropAmountInPaise ?? null,
      channels: alert.channels,
      status: alert.status,
      armed: alert.armed,
      lastTriggeredAt: alert.lastTriggeredAt ?? null,
      createdAt: alert.createdAt,
    };
  }

  export const listAlerts = asyncHandler(async (req, res) => {
    const alerts = await Alert.find({ userId: req.user.id }).sort({ createdAt: -1 });
    return res.status(200).json({ success: true, data: { alerts: alerts.map(toPublicAle
  rt) } });
  });

  export const createAlert = asyncHandler(async (req, res) => {
    const { offerId, type, targetPriceInPaise, dropAmountInPaise, channels } = req.bod
  y;

    const offer = await Offer.findById(offerId);
    if (!offer) {
      return res.status(404).json({
        success: false,
        error: { code: 'NOT_FOUND', message: 'Offer not found.' },
      });
    }

    const alert = await Alert.create({
      userId: req.user.id,
      offerId,
```

```
    type,
    targetPriceInPaise: type === 'target_price' ? targetPriceInPaise : undefined,
    dropAmountInPaise: type === 'fixed_drop' ? dropAmountInPaise : undefined,
    channels,
  });

  return res.status(201).json({ success: true, data: { alert: toPublicAlert(alert) }
});
});

export const updateAlert = asyncHandler(async (req, res) => {
  // CORRECT: find-then-check-ownership, scoped to req.user.id — never trust
  // a client-supplied userId, and never allow a PATCH to change `userId` or
  // `offerId` themselves (not whitelisted below).
  const alert = await Alert.findOne({ _id: req.params.alertId, userId: req.user.id
});

  if (!alert) {
    return res.status(404).json({
      success: false,
      error: { code: 'NOT_FOUND', message: 'Alert not found.' },
    });
  }

  const allowedFields = ['status', 'targetPriceInPaise', 'dropAmountInPaise', 'channe
ls'];
  for (const field of allowedFields) {
    if (req.body[field] !== undefined) {
      alert[field] = req.body[field];
    }
  }

  await alert.save(); // runs full schema validation again, including the
                      // conditional targetPriceInPaise/dropAmountInPaise rule

  return res.status(200).json({ success: true, data: { alert: toPublicAlert(alert) }
});
});

export const deleteAlert = asyncHandler(async (req, res) => {
  const result = await Alert.findOneAndDelete({ _id: req.params.alertId, userId: req.
user.id });
```

```
  if (!result) {
    return res.status(404).json({
      success: false,
      error: { code: 'NOT_FOUND', message: 'Alert not found.' },
    });
  }

  return res.status(200).json({ success: true, data: { deleted: true } });
});
```

```
// server/src/routes/alerts.js
  import { Router } from 'express';
  import { body } from 'express-validator';
  import { requireAuth } from '../middleware/auth.js';
  import { validate } from '../middleware/validate.js';
  import { listAlerts, createAlert, updateAlert, deleteAlert } from '../controllers/ale
  rtController.js';

  const router = Router();

  router.use(requireAuth);

  const createAlertValidation = [
    body('offerId').notEmpty().withMessage('offerId is required.'),
    body('type').isIn(['target_price', 'fixed_drop']).withMessage('type must be "target
  _price" or "fixed_drop".'),
    body('channels')
      .isArray({ min: 1 })
      .withMessage('channels must be a non-empty array.')
      .custom((arr) => arr.every((c) => ['email', 'telegram'].includes(c)))
      .withMessage('channels may only contain "email" and/or "telegram".'),
    // Exactly one of targetPriceInPaise/dropAmountInPaise, matching `type` —
    // Document 1 Section 6.4 and Section 9.
    body('targetPriceInPaise')
      .if(body('type').equals('target_price'))
      .isInt({ min: 1 })
      .withMessage('targetPriceInPaise is required and must be a positive integer for a
  target_price alert.'),
    body('dropAmountInPaise')
      .if(body('type').equals('fixed_drop'))
      .isInt({ min: 1 })
      .withMessage('dropAmountInPaise is required and must be a positive integer for a
  fixed_drop alert.'),
  ];

  router.get('/', listAlerts);
  router.post('/', createAlertValidation, validate, createAlert);
  router.patch('/:alertId', updateAlert);
  router.delete('/:alertId', deleteAlert);

  export default router;
```

H. Beginner explanation: `body('targetPriceInPaise').if(body('type').equals('target_price')).isInt(...)` is `express-validator`'s conditional-validation syntax — it reads "only run this `isInt` rule when `type` in this same request body equals `'target_price'`." This is how the "exactly one of the two price fields, depending on type" rule from Document 1 Section 6.4/9 is enforced at the HTTP boundary, before the Mongoose schema's own conditional `required` function (Section 6.7) enforces it again at the database layer — two independent layers catching the same rule is intentional defense in depth, not redundant.

I. Environment configuration: None additional.

J. Testing: Postman checklist, Section 14.3.

K. Expected results: Matches Document 1 Section 6.4 and the example in Section 6.7 exactly.

L. Troubleshooting: A `PATCH` that silently does nothing usually means the field name in the request body doesn't match one of `allowedFields` exactly (case-sensitive).

M. Acceptance criteria: Creating an alert with both `targetPriceInPaise` and `dropAmountInPaise` present and mismatched `type` is rejected by validation; a second user's `GET /api/alerts` never shows another user's alerts (ACC-03).

N. Git workflow: `feature/alerts` branch (Document 5 Section 5) — note this is a branch not explicitly named in this document's earlier branch list but is the one Document 5's authoritative table assigns to alert work; use it for this task specifically.

O. Completion checklist:

- [ ] `PATCH` never allows changing `userId` or `offerId`
- [ ] Validation enforces exactly one of the two price fields per `type`
- [ ] Every query scoped to `req.user.id`

P. AI-assisted debugging prompt: "Creating an alert with `type: 'target_price'` and no `targetPriceInPaise` is succeeding when it should fail validation. Here's my route's validation array and the request I sent. Identify why the rule isn't firing and the smallest safe fix."

BE-09 — Notification preferences and Telegram link route shell

A. Task ID and title: `BE-09` — Implement `GET /api/notifications`, `GET/PUT /api/notification-preferences`, and the `POST /api/telegram/link` route shell

B. Owner and reviewer: Owner: Member 3 (route/controller shell). Reviewer: Member 1. Depends on: Member 4's `services/notifications/telegram.js` existing (for the real linking logic) — coordinate timing; you can build and merge the shell first with a clearly marked integration point.

C. Purpose: Let a user view their notification history, see/update their notification email, and link their Telegram account.

D. Prerequisites: Auth working; `NotificationEvent` model (BE-02) exists.

E. Exact files: `server/src/routes/notifications.js`, `server/src/routes/notificationPreferences.js`, `server/src/routes/telegram.js`, `server/src/controllers/notificationController.js`, `server/src/controllers/telegramController.js`. **Do not create or edit anything under `server/src/services/notifications/`** — that's Member 4's.

F. Commands: None additional.

G. Implementation:

```
// server/src/controllers/notificationController.js
  import User from '../models/User.js';
  import NotificationEvent from '../models/NotificationEvent.js';
  import { asyncHandler } from '../utils/asyncHandler.js';

  export const listNotifications = asyncHandler(async (req, res) => {
    const events = await NotificationEvent.find({ userId: req.user.id })
      .sort({ createdAt: -1 })
      .limit(100);

    return res.status(200).json({
      success: true,
      data: {
        events: events.map((e) => ({
          id: e._id.toString(),
          alertId: e.alertId.toString(),
          channel: e.channel,
          status: e.status,
          message: e.message,
          errorMessage: e.errorMessage ?? null,
          createdAt: e.createdAt,
        })),
      },
    });
  });

  export const getNotificationPreferences = asyncHandler(async (req, res) => {
    const user = await User.findById(req.user.id);
    if (!user) {
      return res.status(404).json({ success: false, error: { code: 'NOT_FOUND', messag
  e: 'User not found.' } });
    }

    return res.status(200).json({
      success: true,
      data: {
        email: user.email,
        telegramChatId: user.telegramChatId ?? null,
        telegramLinked: Boolean(user.telegramChatId),
      },
    });
  });
```

```
export const updateNotificationPreferences = asyncHandler(async (req, res) => {
  const user = await User.findById(req.user.id);
  if (!user) {
    return res.status(404).json({ success: false, error: { code: 'NOT_FOUND', messag
e: 'User not found.' } });
  }

  // Per the UI spec (Document 1 Section 13.11), the notification email is
  // tied to the account email for this MVP — this endpoint exists per the
  // API contract (Document 1 Section 6.5) but there is intentionally no
  // separate notification-only email field on the User model to update.
  if (req.body.email !== undefined) {
    user.email = req.body.email;
    await user.save();
  }

  return res.status(200).json({
    success: true,
    data: {
      email: user.email,
      telegramChatId: user.telegramChatId ?? null,
      telegramLinked: Boolean(user.telegramChatId),
    },
  });
});
```

```
// server/src/controllers/telegramController.js
  import User from '../models/User.js';
  import { asyncHandler } from '../utils/asyncHandler.js';
  // INTEGRATION POINT: the actual code that validates a link code against
  // Telegram and resolves it to a chat id lives in
  // server/src/services/notifications/telegram.js, owned by Member 4. This
  // controller is a thin shell that calls into it — do not implement Telegram
  // API logic here yourself.
  // import { resolveLinkCode } from '../services/notifications/telegram.js';

  export const linkTelegram = asyncHandler(async (req, res) => {
    const { linkCode } = req.body;

    if (!linkCode) {
      return res.status(400).json({
        success: false,
        error: { code: 'VALIDATION_ERROR', message: 'linkCode is required.' },
      });
    }

    // const chatId = await resolveLinkCode(linkCode, req.user.id); // Member 4's servi
  ce
    // if (!chatId) {
    //   return res.status(400).json({
    //     success: false,
    //     error: { code: 'VALIDATION_ERROR', message: 'That code is invalid or has exp
  ired.' },
    //   });
    // }
    //
    // await User.findByIdAndUpdate(req.user.id, { telegramChatId: chatId });

    // Placeholder response until Member 4's service above is wired in:
    return res.status(501).json({
      success: false,
      error: {
        code: 'SERVER_ERROR',
        message: 'Telegram linking is not wired up yet — see services/notifications/tel
  egram.js (Member 4).',
      },
    });
  });
```

```
// server/src/routes/notifications.js
  import { Router } from 'express';
  import { requireAuth } from '../middleware/auth.js';
  import { listNotifications } from '../controllers/notificationController.js';

  const router = Router();
  router.use(requireAuth);
  router.get('/', listNotifications);
  export default router;
```

```
// server/src/routes/notificationPreferences.js
  import { Router } from 'express';
  import { requireAuth } from '../middleware/auth.js';
  import { getNotificationPreferences, updateNotificationPreferences } from '../control
  lers/notificationController.js';

  const router = Router();
  router.use(requireAuth);
  router.get('/', getNotificationPreferences);
  router.put('/', updateNotificationPreferences);
  export default router;
```

```
// server/src/routes/telegram.js
  import { Router } from 'express';
  import { body } from 'express-validator';
  import { requireAuth } from '../middleware/auth.js';
  import { validate } from '../middleware/validate.js';
  import { linkTelegram } from '../controllers/telegramController.js';

  const router = Router();
  router.use(requireAuth);
  router.post('/link', [body('linkCode').notEmpty()], validate, linkTelegram);

  export default router;
```

> ***The `POST /api/telegram/link-code` addition:*** *Document 4 (Section 12) proposes a companion endpoint, `POST /api/telegram/link-code` (auth required, no body, returns `{ linkCode, botUsername, expiresAt }`), as the first half of the linking flow — the frontend calls this to generate the short code a user then sends to the bot via `/start <code>`, before calling your `POST /api/telegram/link` with that same code.* ***This is Member 4's addition, flagged to Member 1 for Document 1*** *— it is not yet part of Document 1 Section 6.5's official contract as written. Do not implement `POST /api/telegram/link-code` yourself yet; wait for Member 1 to confirm it's been added to Document 1, then add a thin route/controller shell for it here (in your `routes/telegram.js` / `controllers/telegramController.js`, since these files are yours per Document 1 Section 14), following the same "shell calls into Member 4's service" pattern shown above for `/link`.*

H. Beginner explanation: The `501 SERVER_ERROR` placeholder in `linkTelegram` is intentional and temporary — it lets you merge a working, testable route shell (correct validation, correct auth, correct response envelope) before Member 4's `telegram.js` service exists, without either blocking on them or inventing Telegram API logic yourself. Once they deliver `resolveLinkCode` (or whatever they name it), uncomment the real logic and delete the placeholder.

I. Environment configuration: None additional (Telegram's own env vars belong to Member 4).

J. Testing: `GET /api/notification-preferences` with a valid token → `200` with `telegramLinked: false` for a fresh user. `POST /api/telegram/link` with any `linkCode` → `501` until Member 4's service is wired in (expected, not a bug, until that integration lands).

K. Expected results: Matches Document 1 Section 6.5 field-for-field once fully wired.

L. Troubleshooting: If `PUT /api/notification-preferences` 500s, check that `req.body.email` is actually valid per the `User` schema's email `match` regex (Section 6.2) before assuming it's a routing bug.

M. Acceptance criteria: `GET /api/notifications` only ever returns the authenticated user's own events (ACC-03); the Telegram route shell exists, is correctly authenticated and validated, and clearly documents its integration point rather than faking success.

N. Git workflow: `feature/notification-settings` branch (Document 5 Section 5 — this branch is explicitly shared across Members 2, 3, and 4 for this feature area).

O. Completion checklist:

- [ ] `GET /api/notifications` scoped to `req.user.id`
- [ ] `telegramLinked` is computed from `Boolean(user.telegramChatId)`, never hardcoded
- [ ] The integration point for Member 4's Telegram service is clearly commented, not silently faked as working

P. AI-assisted debugging prompt: "My `/api/notification-preferences` PUT route isn't saving the new email. Here's my controller and the exact request body I sent. Identify the root cause and the smallest safe fix."

---

12. Enforcing per-user data isolation

This is the single most important security rule in the whole backend (Document 1 Section 8), and the easiest one to accidentally get wrong under time pressure.

The rule: every query under `/api/watchlist`, `/api/alerts`, `/api/notifications`, and `/api/notification-preferences` must be scoped to `req.user.id` — the id Express attached from the **verified JWT** (Section 8.3) — and **never** to a `userId` the client sent in the request body or query string. A client can put literally anything in a request body; a `userId` field there proves nothing about who is actually making the request. The JWT's `sub` claim, verified by `jwt.verify` against your server's own secret, is the only trustworthy source of "who is this."

Wrong vs. right

```
// WRONG — a bug. Any logged-in user could pass {"userId": "<someone else's id>"}
  // in the request body and read or modify that other user's alerts.
  export const listAlertsWRONG = asyncHandler(async (req, res) => {
    const alerts = await Alert.find({ userId: req.body.userId });
    res.json({ success: true, data: { alerts } });
  });
```

```
// RIGHT — scoped to the id the server itself verified via the JWT.
  export const listAlertsRIGHT = asyncHandler(async (req, res) => {
    const alerts = await Alert.find({ userId: req.user.id });
    res.json({ success: true, data: { alerts: alerts.map(toPublicAlert) } });
  });
```

The same rule applies to single-document lookups for update/delete — notice this pattern, used throughout Section 11:

```
// RIGHT — a single query that both finds the right document AND enforces
  // ownership at the same time. If the alertId exists but belongs to someone
  // else, this returns null, exactly as if it didn't exist at all — which is
  // also why the response is a generic 404, not a 403 that would confirm to
  // an attacker that the resource exists under someone else's account.
  const alert = await Alert.findOne({ _id: req.params.alertId, userId: req.user.id });
```

Grep your own code for `req.body.userId` and `req.query.userId` before every PR — if either ever appears anywhere under your routes/controllers, it's a bug (Document 1 ACC-03 exists specifically to catch this).

---

13. Error handling conventions

13.1 `server/src/middleware/errorHandler.js`

```
// server/src/middleware/errorHandler.js
  export function errorHandler(err, req, res, next) {
    console.error('[error]', err);

    // Mongoose validation errors get a clear, specific message instead of a
    // generic 500 — these are effectively a late-caught validation problem
    // (e.g. a direct .save() call that bypassed express-validator).
    if (err.name === 'ValidationError') {
      const firstMessage = Object.values(err.errors)[0]?.message || 'Validation faile
  d.';
      return res.status(400).json({
        success: false,
        error: { code: 'VALIDATION_ERROR', message: firstMessage },
      });
    }

    // Malformed ObjectId passed in a route param (e.g. /api/alerts/not-a-real-id).
    if (err.name === 'CastError') {
      return res.status(400).json({
        success: false,
        error: { code: 'VALIDATION_ERROR', message: `Invalid id: ${err.value}` },
      });
    }

    // MongoDB duplicate-key error not already caught closer to the source.
    if (err.code === 11000) {
      return res.status(409).json({
        success: false,
        error: { code: 'CONFLICT', message: 'This record already exists.' },
      });
    }

    const status = err.statusCode || 500;
    const isProduction = process.env.NODE_ENV === 'production';

    return res.status(status).json({
      success: false,
      error: {
```

```
        code: err.code && typeof err.code === 'string' ? err.code : 'SERVER_ERROR',
        // Never leak internals (stack trace, file paths) to the client in
        // production — Document 1 Section 8.
        message: isProduction && status === 500 ? 'Something went wrong. Please try aga
in.' : err.message,
      },
    });
  }
```

13.2 `server/src/utils/asyncHandler.js` (repeated here for completeness — see Section 10.1)

```
// server/src/utils/asyncHandler.js
  export function asyncHandler(fn) {
    return function (req, res, next) {
      Promise.resolve(fn(req, res, next)).catch(next);
    };
  }
```

Why this exists: without it, every single `async` controller function would need its own `try { ... } catch (err) { next(err) }` wrapper, or an unhandled rejection inside an `async` Express route handler would hang the request forever (Express 4 does not automatically catch rejected Promises from an `async` function — this is a well-known Express 4 gotcha, fixed only in Express 5). `asyncHandler(fn)` wraps any `async (req, res) => {...}` controller once, at the point it's exported, so every `await SomeModel.find(...)` that throws is automatically funneled into `next(err)` → your `errorHandler` — with zero repeated boilerplate in the controller itself. This is why every controller in this document is written as `export const someController = asyncHandler(async (req, res) => { ... });`.

Centralized error handling, restated: `errorHandler` is the **only** place in the entire codebase that formats an error response. A controller never writes its own `res.status(500).json(...)` for an unexpected failure — it either handles an *expected* failure explicitly (like the `409` for a duplicate email in `register`, Section 10.2) or lets an unexpected one flow through `asyncHandler` → `errorHandler`.

---

14. Testing

14.1 `server/tests/` structure

```
server/tests/
  ├── models.test.js       # schema validation tests (Section 11, BE-02)
  ├── auth.test.js         # register/login/me integration test (below)
  └── setup.js             # shared test-database connection helper
```

14.2 A pragmatic test-database choice

A real production project would run tests against a completely separate database (or an in-memory one via `mongodb-memory-server`) so tests never touch real data and can run in CI without a live Atlas connection. For 4 beginners in 7 days, installing and configuring `mongodb-memory-server` correctly on Windows (it downloads and manages its own MongoDB binary) is extra friction without much payoff at this project's scale. **Pragmatic choice for this project:** add a **second, free Atlas database** on the same cluster, named e.g. `priceradar_test`, and point tests at it via a separate `MONGODB_URI` read from an env var — reusing the same free M0 cluster costs nothing extra and needs no new tooling.

```
// server/tests/setup.js
  import mongoose from 'mongoose';

  export async function connectTestDB() {
    const uri = process.env.TEST_MONGODB_URI || process.env.MONGODB_URI;
    if (mongoose.connection.readyState === 0) {
      await mongoose.connect(uri);
    }
  }

  export async function disconnectTestDB() {
    await mongoose.connection.close();
  }

  export async function clearTestDB() {
    const collections = mongoose.connection.collections;
    for (const key in collections) {
      await collections[key].deleteMany({});
    }
  }
```

Add `TEST_MONGODB_URI` to `server/.env` (and `.env.example` as a placeholder) pointing at the `priceradar_test` database so running tests never touches your real development data.

14.3 `server/tests/auth.test.js` — a real example

```
// server/tests/auth.test.js
  import request from 'supertest';
  import app from '../src/app.js';
  import { connectTestDB, disconnectTestDB, clearTestDB } from './setup.js';

  beforeAll(async () => {
    await connectTestDB();
  });

  afterEach(async () => {
    await clearTestDB();
  });

  afterAll(async () => {
    await disconnectTestDB();
  });

  describe('Auth flow', () => {
    const testUser = { name: 'Test User', email: 'test@example.com', password: 'passwor
  d123' };

    it('registers a new user without returning passwordHash', async () => {
      const res = await request(app).post('/api/auth/register').send(testUser);
      expect(res.status).toBe(201);
      expect(res.body.success).toBe(true);
      expect(res.body.data.user.email).toBe(testUser.email);
      expect(res.body.data.user.passwordHash).toBeUndefined();
    });

    it('rejects a duplicate email with 409 CONFLICT', async () => {
      await request(app).post('/api/auth/register').send(testUser);
      const res = await request(app).post('/api/auth/register').send(testUser);
      expect(res.status).toBe(409);
      expect(res.body.error.code).toBe('CONFLICT');
    });

    it('logs in with correct credentials and returns a token', async () => {
      await request(app).post('/api/auth/register').send(testUser);
      const res = await request(app)
        .post('/api/auth/login')
        .send({ email: testUser.email, password: testUser.password });
```

```
    expect(res.status).toBe(200);
    expect(res.body.data.token).toBeDefined();
  });

  it('returns the same generic error for a wrong password and an unknown email', asyn
c () => {
    await request(app).post('/api/auth/register').send(testUser);
    const wrongPassword = await request(app)
      .post('/api/auth/login')
      .send({ email: testUser.email, password: 'wrongpassword' });
    const unknownEmail = await request(app)
      .post('/api/auth/login')
      .send({ email: 'nobody@example.com', password: 'whatever123' });

    expect(wrongPassword.status).toBe(401);
    expect(unknownEmail.status).toBe(401);
    expect(wrongPassword.body.error.message).toBe(unknownEmail.body.error.message);
  });

  it('GET /api/auth/me requires a valid token', async () => {
    const noToken = await request(app).get('/api/auth/me');
    expect(noToken.status).toBe(401);
    expect(noToken.body.error.code).toBe('AUTH_REQUIRED');

    const register = await request(app).post('/api/auth/register').send(testUser);
    const login = await request(app)
      .post('/api/auth/login')
      .send({ email: testUser.email, password: testUser.password });
    const token = login.body.data.token;

    const me = await request(app).get('/api/auth/me').set('Authorization', `Bearer
${token}`);
    expect(me.status).toBe(200);
    expect(me.body.data.user.email).toBe(testUser.email);
  });
});
```

Run it with:

```
cd server
  npx jest tests/auth.test.js
```

14.4 Postman manual-test checklist

Method	Path	Body	Expected status
GET	`/api/health`	—	200
POST	`/api/auth/register`	`{name, email, password}`	201
POST	`/api/auth/register` (same email again)	same	409
POST	`/api/auth/login`	`{email, password}` (correct)	200, has `token`
POST	`/api/auth/login`	`{email, password}` (wrong password)	401
GET	`/api/auth/me`	— (with `Authorization: Bearer <token>`)	200
GET	`/api/auth/me`	— (no header)	401
GET	`/api/products/search?q=galaxy`	—	200
GET	`/api/products/:productId`	—	200 (or 404 for bad id)
GET	`/api/offers/:offerId/history`	—	200
GET	`/api/watchlist`	— (auth)	200
POST	`/api/watchlist`	`{offerId}` (auth)	201
POST	`/api/watchlist`	same `offerId` again (auth)	409
DELETE	`/api/watchlist/:watchlistItemId`	— (auth, owner)	200
DELETE	`/api/watchlist/:watchlistItemId`	— (auth, NOT owner — second test user)	404
GET	`/api/alerts`	— (auth)	200
POST	`/api/alerts`	`{offerId, type: "target_price", targetPriceInPaise, channels}` (auth)	201
POST	`/api/alerts`	`{offerId, type: "target_price", channels}` — missing price (auth)	400
PATCH	`/api/alerts/:alertId`	`{status: "paused"}` (auth, owner)	200
DELETE	`/api/alerts/:alertId`	— (auth, owner)	200
GET	`/api/notifications`	— (auth)	200
GET	`/api/notification-preferences`	— (auth)	200

Method	Path	Body	Expected status
PUT	`/api/notification-preferences`	`{email}` (auth)	200
POST	`/api/telegram/link`	`{linkCode}` (auth)	501 until Member 4's service lands

15. Debugging

Symptom	Likely cause	Fix
`ValidationError: Path 'email' is required.` (or similar)	A required field was missing/empty when you called `.create()` / `.save()`	Read the message — it names the exact field. Check the request body actually sent that field, and that `express-validator` rules exist for it so bad input is caught before it ever reaches Mongoose
`MongooseServerSelectionError: connect ETIMEDOUT` / similar	Almost always Atlas **Network Access** doesn't allow your current IP, or `MONGODB_URI` has a typo/wrong password	Re-check Network Access allows `0.0.0.0/0` (Section 2.2); re-copy the connection string from Atlas; confirm the password doesn't contain an unescaped special character
`JsonWebTokenError: jwt malformed`	The `Authorization` header isn't in the exact form `Bearer <token>`, or the token got truncated/corrupted somewhere (e.g. a copy-paste error in Postman)	Log `req.headers.authorization` and compare it character-for-character against what you expect; regenerate the token via `/api/auth/login` and copy it fresh
`JsonWebTokenError: invalid signature`	The token was signed with a different `JWT_SECRET` than the one currently in `.env` (e.g. you changed `JWT_SECRET` after issuing the token, or you're testing against a different running server instance)	Log in again to get a fresh token signed with the current secret
CORS error in the browser console (`has been blocked by CORS policy`)	`CLIENT_ORIGIN` in `server/.env` doesn't exactly match the frontend's actual origin (including protocol and port)	Compare `CLIENT_ORIGIN` against the browser's address bar exactly — `http://localhost:5173` is not the same origin as `http://127.0.0.1:5173` or `https://localhost:5173`
`Error: listen EADDRINUSE: address already in use :::5000`	Something (often a previous, not-fully-stopped `nodemon` process) is already listening on port 5000	Find and stop it: `Get-Process -Id (Get-NetTCPConnection -LocalPort 5000).OwningProcess | Stop-Process` in PowerShell, or just pick a different `PORT` temporarily

16. Git workflow

Use exactly the branches finalized in Document 5 Section 5: `feature/project-scaffold`, `feature/user-authentication`, `feature/product-search`, `feature/dashboard-watchlist` cover the bulk of your Day 1–4 work; `feature/alerts` and `feature/notification-settings` (also in Document 5's authoritative table, shared with Member 2 and, for the latter, Member 4) cover BE-08 and BE-09 specifically. Don't invent new branch names.

Full Git mechanics — cloning, resolving conflicts, PR review etiquette, recovering from mistakes — are in Document 5; this is the pattern specific to your work, shown once in full for `feature/user-authentication` (BE-04/BE-05). The same sequence applies to every other branch above — just swap the branch name and the files you `git add`.

```
# 1. Start from an up-to-date main
  git switch main
  git pull --ff-only origin main

  # 2. Create your feature branch
  git switch -c feature/user-authentication

  # 3. ... implement auth.js (middleware), validate.js, routes/auth.js,
  #        controllers/authController.js, User.js ...
  #        ... run your tests ...
  cd server
  npx jest tests/auth.test.js
  cd ..

  # 4. Inspect before staging — never blindly `git add -A`
  git status
  git diff

  # 5. Stage only the files this feature actually touched
  git add server/src/middleware/auth.js server/src/middleware/validate.js `
          server/src/routes/auth.js server/src/controllers/authController.js `
          server/src/models/User.js server/tests/auth.test.js

  # 6. Double-check exactly what you're about to commit
  git diff --cached

  # 7. Commit with a conventional commit message
  git commit -m "feat(auth): implement register, login, me, and JWT middleware"

  # 8. Push and open a PR
  git push -u origin feature/user-authentication
```

Then open the pull request on GitHub (Document 5 Section 6.7 walks through the exact screen), describe what changed, and request review from Member 1 — auth-sensitive code always gets lead review per Document 1 Section 14. Never use `git add -A` or `git add .` here: your working tree may also contain in-progress, unrelated changes from another task, and staging everything risks committing something half-finished or, worse, a stray `.env` that somehow isn't ignored yet.

---

17. Acceptance criteria

Tied to Document 1 Section 17's backend-relevant items, plus backend-specific checks:

- [ ] **ACC-03** — A second registered user's `GET /api/watchlist` and `GET /api/alerts` never return the first user's data (verified with two real Postman-created accounts, not assumed).
- [ ] **ACC-07** — `GET /api/health` returns `200 {status:"ok", time:...}` locally and, later, from the deployed URL.
- [ ] **ACC-09** — `git log -p -- '**/.env'` on the repo shows `.env` was never committed, in either `server/` or anywhere else.
- [ ] Every model's field names, types, `enum` values, `required` rules, and indexes match Document 1 Section 5 exactly — no renamed fields, no missing compound indexes.
- [ ] Every route's method, path, auth requirement, request body, and response shape match Document 1 Section 6 exactly.
- [ ] `passwordHash` is never present in any API response, under any code path (register, login, me, or any error path).
- [ ] Every route that accepts a body runs through `express-validator` rules and the shared `validate` middleware — no controller trusts unvalidated input.
- [ ] `errorHandler.js` is the only place in the codebase that formats an error response envelope; no controller hand-writes its own `{success:false, error:{...}}` for an *unexpected* failure (expected ones like `409 CONFLICT` for a duplicate may still be written explicitly in the controller, as shown throughout Section 11 — that's a deliberate, expected error, not an unexpected one).
- [ ] `server/.env.example` lists every variable from Document 1 Section 7.1 with placeholder values only.
- [ ] CORS is configured from `CLIENT_ORIGIN`, never `origin: "*"`.
- [ ] `routes/monitor.js` and the monitor controller were never created or edited by you — that stays Member 1's.
- [ ] `server/src/providers/` and `server/src/services/notifications/` contain nothing written by you.